Polymorphism



Learning Objectives

After learning this chapter, the students will be able to

- Understand the purpose of overloading
- Construct C++ programs using function, constructor and operator overloading
- Execute and debug programs which contains the concept of polymorphism

15.1 Introduction

The word polymorphism means many forms (poly – many, morph – shapes) Polymorphism is the ability of a message or function to be displayed in more than one form. In C++, polymorphism is achieved through function overloading and operator overloading. The term overloading means a name having two or more distinct meanings. Thus an **'overloaded function'** refers to a function having more than one distinct meaning.

15.2 Function overloading

The ability of the function to process the message or data in more than one form is called as function overloading. In other words function overloading means two or more functions in the same scope share the same name but their parameters are different. In this situation, the functions that share the same name are said to be overloaded and the process is called function overloading . The number and types of a function's parameters are called the **function's signature.** When you call an overloaded function, the compiler determines the most appropriate definition to use, by comparing the argument types you have used to call the function with the parameter types specified in the definitions. The process of selecting the most appropriate overloaded function or operator is called **overload resolution**

15.2.1 Need For Function overloading

Sometimes it's hard to find many different meaningful names for a single action.

Consider the situation to find the area of circle ,triangle and rectangle the following function prototype is given

```
float area_circle(float radius)                          // to calculate the area of a circle
float area_triangle(float half,floatbase,float height)   // to calculate the area of a triangle
float area_rectangle(float length , float breadth)       // to calculate the area of a rectangle
```


Chapter 15 Page 249-259.indd 249 3/24/2020 9:23:24 AM